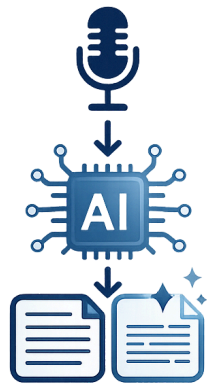


AI-Powered Meeting Assistant

**Automatic Speech Transcription and Meeting Summarization using
Whisper, Llama 2, and Gradio**



**AI-Powered
Meeting Assistant**

Nikolaos Belimpasakis
MEng in Computer Engineering and Informatics

Components

1. Overview and Objectives	2
2. Environment Setup and Dependency Installation	3
3. Speech-to-Text Conversion with OpenAI Whisper	5
4. Audio Transcription Web Application with Gradio	7
5. LLM Integration with IBM WatsonX	12
6. Final Application Implementation	14
7. Conclusion	23
8. References	24

1. Overview and Objectives

The rapid growth of Artificial Intelligence technologies has enabled the development of intelligent applications capable of understanding, processing, and generating human language with remarkable accuracy. Among these advancements, speech recognition and large language models have emerged as powerful tools for transforming unstructured spoken information into meaningful and actionable knowledge.

This project, developed as part of the IBM Generative AI Engineering Specialization [1], aims to design and implement an AI-Powered Meeting Assistant capable of automatically transcribing and summarizing spoken discussions. The application combines state-of-the-art speech recognition technology with Generative AI capabilities to assist users in capturing, organizing, and understanding the content of meetings more efficiently.

The proposed system utilizes OpenAI Whisper [2] to convert audio recordings into textual transcripts and leverages a Large Language Model (Llama 2 [3] through IBM watsonx [4]) to generate concise summaries and extract key discussion points. To provide an intuitive and accessible user experience, the application incorporates a web-based interface developed with Gradio [5], while the final solution is deployed using IBM Code Engine [6].

The primary objectives of the project are:

- To implement an automatic speech recognition system capable of converting spoken language into accurate textual transcripts.
- To utilize Generative AI models for summarizing meeting discussions and extracting the most important information.
- To develop a user-friendly interface that enables seamless interaction with the application.
- To demonstrate the practical integration of multiple AI technologies within a single end-to-end solution.
- To deploy the application in a cloud environment, enabling online accessibility and scalability.

Through the implementation of this project, the practical value of Generative AI in business communication, information management, and productivity enhancement is demonstrated. The resulting application showcases how modern AI technologies can significantly reduce the effort required to document meetings while improving information accessibility and knowledge retention.

2. Environment Setup and Dependency Installation

Before implementing the AI-powered meeting assistant, it is necessary to prepare the development environment and install all required software dependencies. The application combines multiple Artificial Intelligence technologies, including speech recognition, large language models, web interface development, and cloud deployment tools. Therefore, a properly configured Python environment is essential for ensuring compatibility between all components.

The first step involves creating an isolated Python virtual environment. Using a virtual environment prevents dependency conflicts with other projects and allows all required libraries to be installed independently.

```
pip3 install virtualenv
```

```
virtualenv my_env
```

```
source my_env/bin/activate
```

Once the virtual environment is activated, the required Python packages can be installed. These libraries provide support for speech-to-text conversion, large language model integration, web application development, and communication with IBM watsonx services.

```
pip install --force-reinstall "setuptools<70" \
```

```
transformers==4.36.0 \
```

```
torch==2.1.1 \
```

```
gradio==5.23.2 \
```

```
langchain==0.0.343 \
```

```
ibm_watson_machine_learning==1.0.339 \
```

```
openai-whisper
```

In addition to the Python libraries, the FFmpeg multimedia framework [\[7\]](#) must be installed. Whisper relies on FFmpeg to process and decode audio files before transcription. Without FFmpeg, audio recordings cannot be properly loaded and converted into text.

The installation can be performed using the following commands:

```
sudo apt update
```

```
sudo apt install ffmpeg -y
```

After completing these steps, the development environment is fully prepared for implementing the meeting assistant application. The system is now capable of processing audio recordings, performing speech-to-text conversion through OpenAI Whisper, interacting with Large Language Models for text summarization, and supporting the development of a web-based user interface using Gradio.

Installed Technologies

- OpenAI Whisper – Automatic Speech Recognition (ASR) system used for converting spoken language into text.
- Transformers – Hugging Face library providing access to modern pretrained AI models.
- PyTorch – Deep learning framework used for model execution and inference.
- Gradio – Framework for building interactive web interfaces for AI applications.
- LangChain – Library for integrating and orchestrating Large Language Models.
- IBM watsonx AI SDK – Enables communication with IBM's generative AI services.
- FFmpeg – Audio and multimedia processing framework required by Whisper.

This environment configuration serves as the foundation for the implementation of the AI-Powered Meeting Assistant developed throughout the remainder of the project.

The entire project is developed and executed within the IBM Skills Network Lab environment [8]. This cloud-based platform provides the necessary tools, libraries, and computational resources required for implementing, testing, and deploying the AI-powered meeting assistant application.

3. Speech-to-Text Conversion with OpenAI Whisper

The first step of the implementation involves downloading a sample audio recording from a remote URL. The `requests` library is used to send an HTTP GET request and retrieve the audio file. If the request is successful, the file is stored locally under the name `downloaded_audio.mp3`, making it available for the speech recognition process.

```
1 import requests
2
3 # URL of the audio file to be downloaded
4 url = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMSkillsNetwork-GPX004CEEN/Testing%20speech%20to%20text.mp3"
5
6 # Send a GET request to download the file
7 response = requests.get(url)
8
9 # Local file name
10 audio_file_path = "downloaded_audio.mp3"
11
12 # Save the file if the download was successful
13 if response.status_code == 200:
14     with open(audio_file_path, "wb") as file:
15         file.write(response.content)
16     print("File downloaded successfully")
17 else:
18     print("Failed to download the file")
```

The script begins by importing the `requests` library, which enables communication with web servers through HTTP requests. The audio file URL is then specified and a GET request is sent to retrieve its contents. If the server responds successfully (status code 200), the binary content of the response is written to a local MP3 file. Otherwise, an error message is displayed.

After downloading the audio file, the next step is to transcribe its spoken content into text using OpenAI Whisper. The model is accessed through the Hugging Face Transformers pipeline, which provides a simplified interface for Automatic Speech Recognition (ASR) tasks.

```

1  import torch
2  from transformers import pipeline
3
4  # Initialize the speech-to-text pipeline
5  pipe = pipeline(
6      "automatic-speech-recognition",
7      model="openai/whisper-tiny.en",
8      chunk_length_s=30,
9  )
10
11 # Audio file
12 sample = "downloaded_audio.mp3"
13
14 # Generate transcription
15 prediction = pipe(sample, batch_size=8)["text"]
16
17 # Display transcription
18 print(prediction)

```

The `pipeline()` function creates a ready-to-use speech recognition system using the Whisper Tiny English model. The parameter `chunk_length_s=30` divides long audio recordings into smaller segments of 30 seconds, allowing efficient processing.

The downloaded audio file is then provided as input to the model. Whisper analyzes the speech signal and generates a textual transcription, which is returned in the `"text"` field of the output dictionary. Finally, the transcript is printed to the terminal, demonstrating the successful conversion of speech into readable text.

After executing the scripts, the system successfully downloads the audio file and converts the spoken content into text. The generated transcript represents the first stage of the AI-Powered Meeting Assistant pipeline and serves as the input for the subsequent summarization process using Large Language Models.

```

(my_env) theia@theiadocker-nikolaosbeli:/home/project$ python3 simple_speech2text.py
/home/project/my_env/lib/python3.10/site-packages/huggingface_hub/file_download.py:795: FutureWarning:
`resume_download` is deprecated and will be removed in version 1.0.0. Downloads always resume when possible.
If you want to force a new download, use `force_download=True`.
  warnings.warn(
Special tokens have been added in the vocabulary, make sure the associated word embeddings are fine-tuned or trained.
Hello, I want to speak fast because I want to test four speech-to-text applications. Today, whether it's sunny, with a slight breeze, making it perfect for outdoor activity, later I plan for a busy local part, maybe even a picnic. The test is designed to assess the accuracy and responsiveness of the speech-to-text feature. Thank you for participating in this test.
(my_env) theia@theiadocker-nikolaosbeli:/home/project$

```

4. Audio Transcription Web Application with Gradio

Overview

After successfully implementing speech-to-text conversion using the Whisper model, the next step is to integrate this functionality into a user-friendly web interface. Instead of executing the transcription process through the terminal and manually specifying audio files, a graphical application is developed using the Gradio framework.

The purpose of this step is to allow users to upload an audio file directly through a web browser and automatically receive the transcribed text. The application combines the automatic speech recognition capabilities of OpenAI Whisper with Gradio's interactive interface components, resulting in a simple and accessible speech-to-text tool.

This stage represents the first complete version of the meeting assistant application, as it transforms the previously developed transcription script into an interactive web-based solution.

Implementation

A new Python file named `speech2text2_app.py` is created. The application consists of two main components:

1. The speech recognition function that performs audio transcription using Whisper.
2. The Gradio interface that enables interaction between the user and the speech recognition model.

The complete implementation is presented below:

```
import torch
from transformers import pipeline
import gradio as gr

def transcript_audio(audio_file):

    pipe = pipeline(
        "automatic-speech-recognition",
        model="openai/whisper-tiny.en",
        chunk_length_s=30,
    )

    result = pipe(audio_file, batch_size=8)["text"]
    return result

audio_input = gr.Audio(
    sources="upload",
    type="filepath"
)

output_text = gr.Textbox()

iface = gr.Interface(
    fn=transcript_audio,
    inputs=audio_input,
    outputs=output_text,
    title="Audio Transcription App",
    description="Upload the audio file"
)

iface.launch(
    server_name="0.0.0.0",
    server_port=7860
)
```

Code Explanation

The application begins by importing the required libraries.

```
import torch
```

```
from transformers import pipeline
```

```
import gradio as gr
```

The `transformers` library provides access to the Whisper model through the Hugging Face pipeline API, while Gradio is responsible for creating the web interface. PyTorch is automatically used as the backend framework for model inference.

A custom function named `transcript_audio()` is then defined.

```
def transcript_audio(audio_file):
```

This function receives the uploaded audio file as input and returns the transcribed text.

Inside the function, a speech recognition pipeline is initialized:

```
pipe = pipeline(  
    "automatic-speech-recognition",  
    model="openai/whisper-tiny.en",  
    chunk_length_s=30,  
)
```

The pipeline uses the Whisper Tiny English model ([openai/whisper-tiny.en](#)) to perform automatic speech recognition (ASR). The parameter `chunk_length_s=30` instructs the model to process the audio in 30-second segments, which improves efficiency when handling longer recordings.

The uploaded audio file is then passed to the model:

```
result = pipe(audio_file, batch_size=8)["text"]
```

The `batch_size=8` parameter specifies how many audio segments are processed simultaneously. The model returns a dictionary containing several outputs, and the transcription is extracted from the `"text"` field.

The transcription is then returned:

return result

Next, the Gradio input component is created:

```
audio_input = gr.Audio(
```

```
sources="upload",  
type="filepath"  
)
```

This component allows users to upload audio files directly from their local computer. The parameter `type="filepath"` ensures that the uploaded file is passed to the transcription function as a file path.

The output component is defined as a text box:

```
output_text = gr.Textbox()
```

This text box will display the generated transcription to the user.

The interface itself is then created:

```
iface = gr.Interface(  
    fn=transcript_audio,  
    inputs=audio_input,  
    outputs=output_text,  
    title="Audio Transcription App",  
    description="Upload the audio file"  
)
```

The `gr.Interface()` method connects the transcription function with the user interface.

Its parameters specify:

- `fn`: the function executed when the user submits an audio file.
- `inputs`: the audio upload component.
- `outputs`: the text box displaying the transcription.

- **title**: the title displayed at the top of the application.
 - **description**: a short instruction for the user.
-

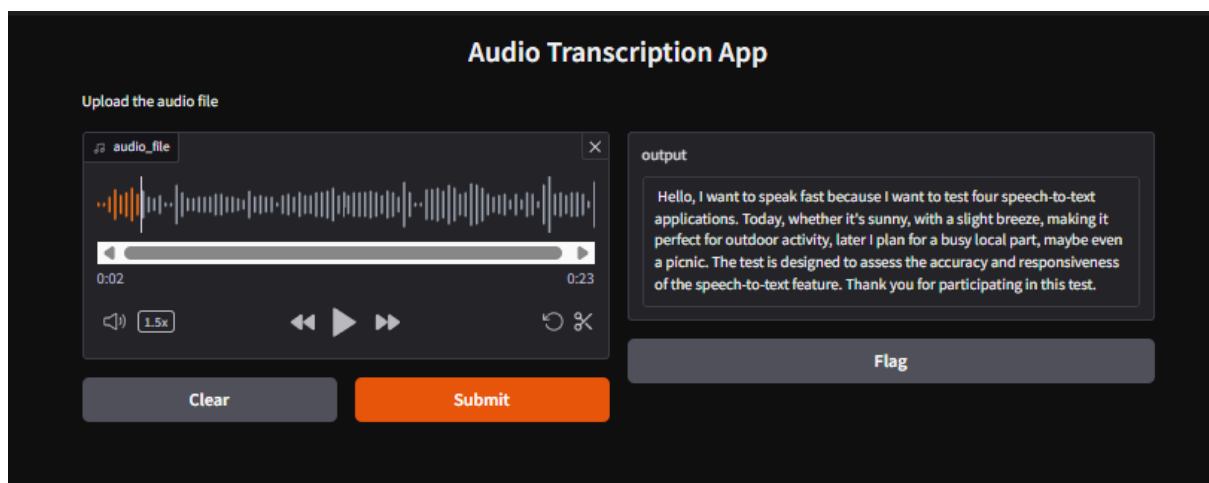
Finally, the application is launched:

```
iface.launch(  
    server_name="0.0.0.0",  
    server_port=7860  
)
```

The **launch()** method starts a local web server and makes the application accessible through a browser. The application listens on port **7860**, which is the default port commonly used by Gradio applications.

Outcome

After executing the script, a web-based audio transcription application becomes available. Users can upload an MP3 audio file through the interface, and the Whisper model automatically converts the speech into text. The generated transcription is displayed instantly within the application, providing a significantly more user-friendly solution compared to command-line execution. This step also establishes the foundation for the next phase of the project, where Large Language Models (LLMs) will be integrated to summarize the transcribed meeting content and extract key discussion points.



5. LLM Integration with IBM WatsonX

After successfully implementing speech-to-text transcription and creating a Gradio-based user interface, the next step is integrating a Large Language Model (LLM) capable of understanding and generating human-like text.

In this stage, IBM WatsonX is utilized to access Meta's Llama 3 model. The objective is to verify that the application can communicate with an advanced language model and generate meaningful responses based on user prompts. This functionality will later be used to summarize meeting transcripts and extract important information from discussions.

A new Python file named **simple_llm.py** is created.

The application uses IBM Watson Machine Learning and LangChain libraries to connect to WatsonX and interact with a foundation model. The service credentials are first defined, followed by the generation parameters that control the model's behavior.

The model is then initialized using the **meta-llama/llama-3-2-11b-vision-instruct** foundation model. Once the model has been configured, it is wrapped inside a LangChain **WatsonxLLM** object, which provides a simplified interface for sending prompts and receiving generated responses.

The following prompt is used to test the integration:

```
print(llm("How to read a book effectively?"))
```

When executed, the model processes the request and returns a detailed text response containing practical suggestions for effective reading.

The parameter "MAX_NEW_TOKENS" defines the maximum number of tokens that can be generated in a single response. Increasing this value allows the model to produce longer and more detailed answers. The "temperature" parameter controls the randomness of the generated output. Lower values generate more predictable and deterministic responses, while higher values encourage more creative and diverse text generation.

The WatsonxLLM class acts as a bridge between LangChain and IBM WatsonX, enabling easy interaction with the selected foundation model through simple Python function calls.

```

1  from ibm watson machine learning.foundation_models import Model
2  from ibm watson machine learning.foundation_models.extensions.langchain import WatsonxLLM
3  from ibm watson machine learning.metanames import GenTextParamsMetaNames as GenParams
4
5  my_credentials = {
6      "url"      : "https://us-south.ml.cloud.ibm.com"
7  }
8
9  params = {
10     GenParams.MAX_NEW_TOKENS: 700, # The maximum number of tokens that the model can ge
11     GenParams.TEMPERATURE: 0.1,   # A parameter that controls the randomness of the tok
12 }
13
14  LLAMA2_model = Model(
15     model_id= 'meta-llama/llama-3-2-11b-vision-instruct',
16     credentials=my_credentials,
17     params=params,
18     project_id="skills-network",
19 )
20
21  llm = WatsonxLLM(LLAMA2_model)
22
23  print(llm("How to read a book effectively?"))

```

After running the script, the model successfully generates a natural language response based on the provided prompt. This confirms that the application can communicate with IBM WatsonX and utilize a Large Language Model for text generation tasks.

The generated output demonstrates the model's ability to understand user instructions and provide coherent, context-aware responses.

```

Reading is one of the most efficient ways to gain knowledge and expand your mind. However, not all reading is created equal. Here are some tips to help you read a book effectively:

1. Set goals: Before you start reading, set specific goals for what you want to achieve. Do you want to learn a new skill or gain a deeper understanding of a particular subject? Having clear goals in mind will help
2. Choose the right book: Not all books are created equal. Choose a book that aligns with your goals and interests. Look for books that have received positive reviews and are written by experts in their field.
3. Create a reading schedule: Set aside dedicated time each day or week to read. Consistency is key to making progress and retaining information.
4. Take notes: Taking notes while reading can help you retain information and engage with the material on a deeper level. Write down key points, questions, and insights that come to mind as you read.
5. Summarize the material: After finishing a chapter or section, summarize the material in your own words. This will help you understand the material better and

```

Although this step only demonstrates a simple text-generation example, it is a critical component of the overall project architecture. In the final meeting assistant application, the LLM will receive the transcript generated by OpenAI Whisper and automatically produce concise meeting summaries, key discussion points, and actionable insights.

6. Final Application Implementation

At this stage, all previously developed components are combined into a single application capable of automatically transcribing and summarizing audio recordings. The final solution integrates OpenAI Whisper for speech recognition, IBM WatsonX Llama 3 for text analysis and summarization, LangChain for prompt management, and Gradio for creating a user-friendly web interface.

The application follows a simple workflow. The user uploads an audio file through the Gradio interface. The audio is first processed by OpenAI Whisper, which converts speech into text. The generated transcript is then forwarded to a Large Language Model hosted on IBM WatsonX. Finally, the model extracts the most important information and generates a structured summary that is displayed directly to the user.

Complete Application Code

```
import torch
import os
import gradio as gr

#from langchain.llms import OpenAI
from langchain.llms import HuggingFaceHub

from transformers import pipeline
from langchain.prompts import PromptTemplate
from langchain.chains import LLMChain

from ibm_watson_machine_learning.foundation_models import Model
from ibm_watson_machine_learning.foundation_models.extensions.langchain import WatsonxLLM
from ibm_watson_machine_learning.metanames import GenTextParamsMetaNames as GenParams

my_credentials = {
    "url": "https://us-south.ml.cloud.ibm.com"
}

params = {
    GenParams.MAX_NEW_TOKENS: 800, # The maximum number of tokens that the model can generate in a single run.
    GenParams.TEMPERATURE: 0.1, # A parameter that controls the randomness of the token generation.
}

LLAMA2_model = Model(
    model_id='meta-llama/llama-3-2-11b-vision-instruct',
    credentials=my_credentials,
    params=params,
    project_id="skills-network",
)

llm = WatsonxLLM(LLAMA2_model)

##### ----- Prompt Template ----- #####
```

```

temp = """
<s><sys>>
List the key points with details from the context:
[INST] The context : {context} [/INST]
<</sys>>
"""

pt = PromptTemplate(
    input_variables=["context"],
    template=temp
)

prompt_to_LLAMA2 = LLMChain(llm=llm, prompt=pt)

#####----- Speech2text -----#####

def transcript_audio(audio_file):
    # Initialize the speech recognition pipeline
    pipe = pipeline(
        "automatic-speech-recognition",
        model="openai/whisper-tiny.en",
        chunk_length_s=30,
    )
    # Transcribe the audio file and return the result
    transcript_txt = pipe(audio_file, batch_size=8)["text"]
    result = prompt_to_LLAMA2.run(transcript_txt)

    return result

#####----- Gradio -----#####

audio_input = gr.Audio(sources="upload", type="filepath")
output_text = gr.Textbox()

iface = gr.Interface(
    fn=transcript_audio,
    inputs=audio_input,
    outputs=output_text,
    title="Audio Transcription App",
    description="Upload the audio file"
)

iface.launch(server_name="0.0.0.0", server_port=7860)

```

Step-by-Step Code Analysis

Importing the Required Libraries

The first section imports all required libraries. Transformers provides access to OpenAI Whisper, LangChain is used to create prompt chains for the language

model, IBM Watson Machine Learning provides access to WatsonX foundation models, and Gradio is responsible for building the web interface.

```
from transformers import pipeline
```

```
from langchain.prompts import PromptTemplate
```

```
from langchain.chains import LLMChain
```

```
import gradio as gr
```

These libraries form the core infrastructure of the application.

Configuring the IBM WatsonX Model

The application connects to IBM WatsonX using predefined credentials and generation parameters.

```
my_credentials = {  
  
    "url": "https://us-south.ml.cloud.ibm.com"  
  
}
```

The generation parameters define how the language model behaves.

```
params = {  
  
    GenParams.MAX_NEW_TOKENS: 800,  
  
    GenParams.TEMPERATURE: 0.1,  
  
}
```

The maximum number of generated tokens is set to 800, allowing the model to produce detailed summaries. The temperature value is kept low to generate consistent and deterministic outputs.

Initializing the Llama 3 Model

The language model is instantiated using the WatsonX SDK.

```
LLAMA2_model = Model(  
  
    model_id='meta-llama/llama-3-2-11b-vision-instruct',  
  
    credentials=my_credentials,  
  
    params=params,  
  
    project_id="skills-network",  
  
)
```

This object represents the foundation model responsible for analyzing the meeting transcript and generating the final summary.

To simplify interaction with the model, it is wrapped using LangChain:

```
llm = WatsonxLLM(LLAMA2_model)
```

Creating the Prompt Template

A prompt template is created to guide the model's behavior.

```
temp = ""
```

```
<s><<sys>>
```

List the key points with details from the context:

```
[INST] The context : {context} [/INST]
```

```
<</sys>>
```

```
""
```

The template instructs the model to identify the most important points from the provided transcript rather than generating a generic response.

The template is then converted into a PromptTemplate object:

```
pt = PromptTemplate(

    input_variables=["context"],

    template=temp

)
```

Creating the LLM Chain

The prompt template and the language model are combined into a single processing chain.

```
prompt_to_LLAMA2 = LLMChain(

    llm=llm,

    prompt=pt )
```

Whenever a transcript is provided, LangChain automatically inserts the transcript into the prompt and forwards the final request to the language model.

Implementing Speech-to-Text Conversion

The `transcript_audio()` function contains the application's main processing logic.

```
pipe = pipeline(  
  
    "automatic-speech-recognition",  
  
    model="openai/whisper-tiny.en",  
  
    chunk_length_s=30,  
  
)
```

This creates an Automatic Speech Recognition pipeline using OpenAI Whisper Tiny.

The uploaded audio file is then converted into text.

```
transcript_txt = pipe(  
  
    audio_file,  
  
    batch_size=8  
  
)["text"]
```

The generated transcript contains the spoken content of the audio recording.

Generating the Meeting Summary

Once the transcription is complete, the transcript is passed to the language model.

```
result = prompt_to_LLAMA2.run(  
  
    transcript_txt  
  
)
```

The LLM analyzes the transcript and extracts the key discussion points, producing a concise summary that can be easily reviewed by the user.

Building the Gradio Interface

The web interface is created using Gradio.

```
audio_input = gr.Audio(  
  
    sources="upload",  
  
    type="filepath"  
  
)
```

```
output_text = gr.Textbox()
```

The audio component allows users to upload audio recordings, while the textbox displays the generated summary.

The interface is defined as:

```
iface = gr.Interface(  
  
    fn=transcript_audio,  
  
    inputs=audio_input,
```

```
outputs=output_text,  
  
title="Audio Transcription App",  
  
description="Upload the audio file"  
  
)
```

Whenever a user uploads a file and presses **Submit**, the `transcript_audio()` function is executed automatically.

Finally, the application is launched:

```
iface.launch(  
  
    server_name="0.0.0.0",  
  
    server_port=7860  
  
)
```

This starts a local web server and makes the AI Meeting Assistant accessible through a browser.

The final implementation demonstrates how Speech-to-Text systems, Large Language Models, Prompt Engineering, and Web Interfaces can be combined to create a practical AI-powered meeting assistant capable of automatically transforming spoken conversations into structured and actionable information.

7. Conclusion

This project demonstrated the development of an AI-powered meeting assistant capable of automatically transcribing and summarizing spoken discussions. By integrating modern Artificial Intelligence technologies within a single application, an end-to-end solution was created that transforms raw audio recordings into structured and meaningful information.

The implementation combined OpenAI Whisper for speech recognition, IBM WatsonX Llama 3 for natural language understanding and summarization, LangChain for prompt management and model interaction, and Gradio for the development of a user-friendly web interface. The integration of these technologies showcased how Speech-to-Text systems and Large Language Models can work together to automate information processing tasks that traditionally require significant manual effort.

Through the successful completion of the project, practical experience was gained in the use of foundation models, prompt engineering, speech recognition pipelines, and AI application development. The project also highlighted the growing importance of Generative AI in business environments, where automatic meeting documentation and knowledge extraction can improve productivity, reduce administrative workload, and enhance information accessibility.

Overall, the developed application demonstrates a practical real-world use case of Generative AI and Natural Language Processing technologies. The resulting system provides an efficient mechanism for converting spoken conversations into concise summaries and key discussion points, illustrating the potential of AI-driven solutions for supporting communication, collaboration, and knowledge management in modern organizations.

Finally, the project successfully achieved its primary objective of building a complete AI-powered meeting assistant and provided valuable insight into the design, implementation, and deployment of intelligent applications using contemporary AI frameworks and services.

References

1. <https://www.coursera.org/professional-certificates/ibm-generative-ai-engineering>, “IBM Generative AI Engineering”, Coursera.
2. <https://openai.com/el-GR/index/whisper/>, “Introducing Whisper”, OpenAI.
3. https://huggingface.co/docs/transformers/model_doc/llama2, “Llama 2 documentation”, Hugging Face.
4. <https://www.ibm.com/docs/en/watson-libraries>, “IBM Watson Natural Language Processing Library for Embed”, IBM.
5. <https://gradio.app/docs>, “Gradio API Documentation”,
6. <https://www.ibm.com/products/code-engine>, “IBM Cloud Code Engine”, IBM.
7. <https://ffmpeg.org/ffmpeg.html>, “ffmpeg Documentation”.
8. <https://skills.network/lab-tools>, “Skills Network Labs”, IBM.